

LingBot-VLA 2.0 优化开关表: 逐项启动验证

pr3967-combined 分支全部新增开关的默认值、作用与真机启动结果

测试平台:8×A100-80GB(liuyue 机) 代码:lerobot pr3967-combined 环境:conda lvla-lerobot

摘要 本文对 pr3967-combined 分支为 LingBot-VLA 2.0 引入的全部优化开关做**逐项真机启动验证**: 训练开关经 lerobot-train 真实启动 (steps=3、gpudat SO101 真实数据集), 推理开关经 ckpt 加载与 predict_action_chunk(rollout 的策略路径), 另设一臂完整复刻 rollout 加载方式 (开关固化进 config.json 后 from_pretrained)。共 13 臂,**11 臂通过**; 两臂给出负面结论——EFFICIENT_ATTENTION 后端直接无可用 kernel, CUDNN_ATTENTION 能启动但第一个训练步梯度即 NaN。最终纪律一句话:**sdpa_backend 整体不要设, 自动后端是唯一安全且最优的选择**。

关键词: LingBot-VLA 2.0; 开关验证;lerobot-train;lerobot-rollout;SDPA 后端

1 引言

本分支相对上游 PR #3967 引入了一组训练/推理优化开关。开关表不能只靠读代码核对: sdpa_backend 一类开关 “能解析、能启动” 与 “训练可用” 之间隔着真实的数值缺陷。本文回答的问题是: **每一个开关, 真启动能不能起来? 起来之后数值对不对?**

2 验证设置

验证矩阵驱动脚本 bench/run_switch_smoke.sh, 产物在 A100 侧 ~/accel-bench/switch_smoke/ 目录。权重为 robotwin 6B 转换 ckpt, 数据集 gpudat SO101(993 段 / 132k 帧 / 3 相机 512×512)。宿主机 GPU 0-3 被 LIBERO eval 占用, 本矩阵全程跑在 GPU 4-7。

训练臂 (每臂独占一卡):lerobot-train 启动,steps=3、batch_size=1、num_workers=2、save_checkpoint=false, 其余与手册标准命令一致。判据:exit 0 且 3 步 loss 有限、与对照组一致。

推理臂:bench/bench_lingbot_v2.py infer(iters=3, warmup=1), 即 from_pretrained + 合成观测 + predict_action_chunk。I6 为 rollout 完整路径复刻: 五个推理开关固化进 config.json 副本后从头加载。

3 训练开关验证结果

对照组 (T0)3 步 loss 为 0.429/0.446/2.448;optimizer_fused 与 gradient_checkpointing 两臂 loss 逐位贴合, 数值等价性在真训练路径上再确认。CUDNN_ATTENTION 是最危险的一类失败: 进程正常退出、前向 loss 正常, 只有多步训练才暴露梯度 NaN——这正是本分支把它列为禁用项的直接证据。

表 1: 训练开关逐项启动验证 (lerobot-train,steps=3,B=1,A100-80GB 单卡)

开关 (--policy. 前缀)	默认	启动验证与结论
optimizer_fused	false	通过。3 步 loss 0.429/0.446/2.445, 与对照组一致;fused AdamW 此前 e2e 实测 -5.4% step 时间
gradient_checkpointing	false	通过。loss 与对照组逐位一致; 但 B≤4 时 +38% 慢, 显存够就别开
moe_dense_max_tokens	512	通过。=0 回退逐专家循环路径可用 (loss 数值路径不同属预期); 默认开即优化后状态
attention_implementation	sdpa	保持 sdpa 。eager+bf16 经 300 步真实数据 A/B:loss 曲线等价 (max Δ ≈ 0.012) 但稳态 +2.2% 慢、+2.9GB 显存、B=4 OOM——合成 bench 上的 eager 优势是相同样本平铺的假象
sdpa_backend=EFFICIENT_ATTENTION	null	起不来 。RuntimeError: No available kernel——mem-efficient 后端不支持本模型 bool mask + GQA + head_dim 128 的组合
sdpa_backend=CUDNN_ATTENTION	null	能启动但训练即毁 。exit 0、前向正常, 但 step 1 梯度 grdn:nan, step 2 起 loss=nan; 单步梯度对照 1222/1267 个梯度张量 NaN

4 推理开关验证结果

表 2: 推理开关逐项启动验证 (bench infer 臂,I6 为固化后 rollout 加载路径)

开关	默认	启动验证与结论
compile_predict_velocity	false	通过 (I2)。编译需 ≥3 次 warmup, 冒烟 warmup=1 的均值被编译时间污染, 稳态看 I6
compile_predict_velocity_mode	default	通过 (随 I2/I3/I6)。锁定 max-autotune-no-cudagraphs
compile_prefix	false	通过 (I3 全套栈)。需先开 compile_predict_velocity
preprocess_device=cuda	null	通过 (I1)。预处理 519.4 → 198.0 ms(-61.9%); 仅推理, 训练自动留 CPU
num_steps	10	通过 (I4,steps=5)。锁定 10; 改小线性降时延但掉动作质量
moe_dense_max_tokens=0	512	通过 (I5)。关闭后 911.2 → 1129.5 ms(+24.0%), 符合回退预期

对照基线 (I0, 全默认 eager + CPU 预处理): 总时延 911.2 ms, 其中预处理 519.4 ms。

I6:rollout 完整加载路径。将 compile_predict_velocity + max-autotune-no-cudagraphs + compile_prefix + preprocess_device=cuda + num_steps=10 固化进 ckpt 的 config.json 后 from_pretrained: iter0 43.7 s(一次性 inductor 编译),iter1 12.5 s, 稳态 **258.0 ms**, 动作张量 (1, 50, 14) 数值有限。手册 §3a 的固化流程端到端成立, rollout 前做一次即可。

5 讨论

两个负面结果都来自真启动而非代码阅读:EFFICIENT_ATTENTION 在加载后第一次 attention 调用即报无可用的 kernel;CUDNN_ATTENTION 的 NaN 只在 backward 出现, 前向 parity 检查 (rel diff 3.3×10^{-7}) 完全无法发现。后端切换类开关的验收门槛因此定为**多步 e2e 训练**, 单步前向对比不足为据。sdpa_backend 保留在配置里仅供调试, 生产命令一律不写。

另有一次反向教训: 合成 bench(相同样本平铺、序列短且定长) 曾显示 eager+bf16 训练更快更省 (-6.9%、-10GB), 据此一度准备改默认;300 步真实数据集 A/B(2026-08-14, loss 曲线完全等价) 推翻了它——真实变长序列下 eager 稳态 +2.2% 慢、+2.9GB、B=4 OOM。**bench 结论迁移到真实训练前必须用真实数据复核**,sdpa+bf16 维持推荐默认。

6 结论

13 臂矩阵,11 臂通过、2 臂给出禁用级证据。推荐组合不变: **训练全默认** + optimizer_fused=true, 单卡 B=3 / 多卡 B=2 每卡; **推理**五开关固化进 config.json(e2e 436 ms @ robotwin 3 相机, 约 300 ms @ 单相机)。**sdpa_backend 一律不设**。

参考与产物

- [1] 验证矩阵驱动与日志:bench/run_switch_smoke.sh;
A100 侧日志目录 ~/accel-bench/switch_smoke/。
- [2] 开关实现 commit:e23a03f5、4ce46ede、c756a657 (分别为 sdpa_backend 与 fused AdamW、bench 臂扩展)。
- [3] cuDNN 梯度 NaN 单步对照:bench/check_cudnn_grads.py(commit 217a848f)。
- [4] 配套命令手册:docs/accel/lingbot_vla_train_rollout_commands.md。